Static Code Analysis

Lab 5 | Recruitment Tracker | Submission Report

Student	SRN	Section
Debaditya Chakrabarti	PES1UG24CS140	C

1. Objective and outcome

Improve the security, correctness and maintainability of the assigned Python program using Pylint, Bandit and Flake8. Twelve groups of reported issues were corrected, prioritising the two medium-severity security findings. The original demonstration behaviour is preserved.

Acceptance check	Before	Verified final result
Pylint score / messages	4.47/10 · 21 messages	10.00/10 · 0 messages
Bandit findings	2 medium · 1 low · 0 high	0 findings
Flake8 findings	8 findings	0 findings
Regression tests	Not supplied upstream	15 tests passed

Source and environment

Assigned repository: github.com/SETAPESU26/L5_14_recruitment_tracker
Assessed file: recruitment_tracker.py
Source commit: a5dba868a578d85536d8c6b0ba7059961a7b47f0
Analysis date: 28 October 2026

CPython 3.13.15 · Pylint 4.1.3 · Bandit 1.9.4 · Flake8 7.4.1. The virtual environment is pinned in requirements-dev.txt. Reports cover the assigned program only; no rule exclusions or inline suppressions were used.

Method

Inspect and run the original demo → save baseline reports → fix security findings → repair correctness and maintainability → resolve formatting → rerun all tools and regression tests → integrate the submission under Lab5/ without replacing Lab 1 files.

2. Documented issues — security and correctness

Original line references are from the unchanged upstream script. Final line references are from the cleaned script. Bandit severity is preserved; Pylint/Flake8 findings are not assigned invented security ratings.

Issue / tool code	Type	Original line(s)	Description	Fix approach
1. SQL interpolation B608	Security Medium	8	Applicant values formed part of the SQL statement; quotes could break the query.	Use parameterised SQL and separate values. Final lines 16–23.
2. Unrestricted eval B307 / W0123	Security Medium	14	Salary input was executed as arbitrary Python code.	Allowlisted, bounded AST arithmetic preserves the demo without executing code. Final 26–64, 72.
3. Ignored exception B110 / W0718	Security Low / quality	24–29	A broad catch silently hid failed database updates.	Propagate SQLite errors and use a transaction context for rollback. Final 80–86.
4. Mixed return paths R1710	Correctness	12–21	No qualifying applicants produced None instead of a list.	Always return result, including an empty list. Final 67–77.
5. Unused variable W0612 / F841	Maintainability	15	total_seen was assigned but never read.	Remove the dead assignment; no suppression.
6. Default encoding W1514	Portability	32	Export encoding varied across operating systems.	Specify UTF-8 and verify Unicode round trips. Final 91.

Priority rationale: Bandit reported no high-severity findings. The two medium findings were fixed first, followed by the low-severity swallowed exception and the remaining quality/style findings.

3. Documented issues — resources and style

Issue / tool code	Type	Original line(s)	Description	Fix approach
7. Manual file closure R1732	Resource handling	32–35	An export exception could bypass close().	Use a with-open context manager. Final 89–95.
8. Function naming C0103	Maintainability	7, 24, 31, 38	Four helpers used mixed-case names.	Use snake_case and update every caller. Final 16, 80, 89, 98.
9. Missing docstrings C0114 / C0116	Documentation	1 and function definitions	Module and functions had no purpose or behaviour documentation.	Add concise module/function docstrings, including the arithmetic grammar.
10. Long lines C0301 / E501	Style	4, 8, 13, 26, 39	SQL and chained calls exceeded line-length limits.	Wrap statements and SQL strings; meet the default 79-character Flake8 limit.
11. Trailing spaces C0303 / W291	Style	15	The unused-variable line contained trailing whitespace.	Delete the dead line and remove trailing spaces.
12. Function spacing E302	Style	31	Top-level definitions were separated by only one blank line.	Use two blank lines between definitions. Final export definition: 89.

Additional manual-review improvements

CSV export now uses csv.writer to quote commas, quotation marks and embedded newlines correctly. The demo has a __main__ guard to avoid running on import. The script closes the SQLite connection in a finally block. These improvements are not misrepresented as additional scanner findings.

The complete issue table, including exact final line references and verification notes, is included in issues.md.

4. Verification and reproducibility

Regression evidence

All 15 unittest tests passed. Coverage includes the original demonstration and CSV records; salary-budget arithmetic, boundaries and role filtering; empty results; apostrophe-containing input; status updates/counts; propagated database failures and rollback; rejected or unbounded expressions; Unicode/quoted CSV data; export errors; and import behaviour.

```
['Nina']
2 in interview
```

The final demo output matches the original. The export retains three records, three columns and no header. The database remains in-memory, as in the assigned program.

Repeat the final checks

```
cd Lab5
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-dev.txt
bash run_analysis.sh
```

The script saves the three scanner reports and test report in reports/after/, records exit statuses, and fails if any check fails. All recorded exit statuses are zero. The empty final Flake8 report is an expected successful result. Baseline reports remain in reports/before/.

Three focused fixing commits

1. Bandit: parameterise SQL, replace eval, and stop silencing exceptions.

2. Pylint: fix returns, naming, documentation and resource handling.

3. Flake8: fix final line length/spacing and add verified submission files.

Use git log -3 --oneline to inspect the commits. The assigned code is integrated into the existing Lab repository as Lab5/; upstream provenance is recorded rather than importing its unrelated Git history.

Scope and limitations

Clean scans are not proof of defect-free or fully secure software. Tool counts overlap and are not a unique-defect count. The salary evaluator deliberately accepts only bounded numeric arithmetic. Database persistence and spreadsheet-formula neutralisation were not added to this lab application.

5. Reflection

Prompt clarification: the supplied handout mentions reflection questions but does not list them. The following are proposed lab-aligned prompts, not quoted questions. Detailed responses are also provided in reflection.md for review and personalisation.

1. What did each tool contribute?

Pylint identified quality and correctness concerns, including inconsistent returns, broad exception handling, unused variables, naming and file management. Bandit highlighted interpolated SQL, eval and swallowed exceptions. Flake8 found formatting defects and the unused local. Their overlapping findings provided corroboration, but each tool contributed a different perspective; message totals therefore should not be summed as unique defects.

2. Which fix needed the most care?

The medium-severity SQL and eval findings were prioritised. Parameterised SQL was a direct correction. Replacing eval needed more care because the original demo intentionally adds two salary values. A simple integer conversion would break it, and ast.literal_eval does not evaluate that addition. An allowlisted AST evaluator retained basic arithmetic while rejecting calls, names, attributes and unsupported operators, with expression-length and numeric bounds.

3. How were fixes verified, and what remains uncertain?

Default scans reached Pylint 10.00/10 with no Bandit or Flake8 findings. Fifteen tests also exercised normal workflows and repaired edge cases. Dynamic checks were important because static tools did not identify every concern, such as CSV quoting. A clean scan cannot establish all business rules or runtime safety. The application retains its original in-memory storage and is not claimed to be a production-ready applicant system.

4. How should this fit into future work?

Run scanners and regression tests before commits and in continuous integration. Keep dependencies pinned, preserve baseline evidence and make focused commits that are easy to review. Review findings in context, prioritise security and correctness, and fix causes rather than broadly suppressing warnings. Tests should accompany behavioural changes so a better style score does not hide a regression.

Conclusion

The final program satisfies the lab thresholds in the recorded environment: more than four issues fixed, Pylint above 9.5, all Bandit findings resolved and no Flake8 findings. The submission includes the cleaned code, traceable issue documentation, reflections, before/after reports and regression evidence.